CMPSC 448
Midterm Project

Who Wrote It?
Identifying LLM Families from Their Responses

Doucheng Peng (Team leader)
dbp5625@psu.edu

Hongchang Ma (Team member)

Fall 2026

448

Contents

1. What we wanted to find out **3**

2. Building the dataset **4**

3. Models and training **5**
(a) Representing the text . . . 5
(b) Two simple baselines . . . 5
(c) CNN . . . 5
(d) LSTM . . . 5
(e) Training and evaluation . . . 6

4. RQ1: Can we recognize the family? **7**

5. RQ2: Does the prompt help? **10**

6. RQ3: Does recognition carry over to another task? **11**
6.1 Trying newer models . . . 14

7. RQ4: What might the classifiers be recognizing? **16**
(a) A few things we can measure . . . 16
(b) Looking at actual tokens . . . 17
(c) Removing possible clues . . . 18
(d) Our working guess . . . 19
(e) Three simple follow-up guesses . . . 19

8. What happens without labels, or with less data? **20**
(a) K-means . . . 20
(b) Less training data . . . 21

9. What we take from this **22**

References **23**

Appendix A: Model versions **24**

Appendix B: Feature and cluster details **25**

Appendix C: All network runs **26**
LSTM runs . . . 27

1. What we wanted to find out

Take two answers to the same question. Even without the model names, sometimes one of them feels more like a particular model: it starts with a familiar phrase, uses a certain kind of list, or keeps adding explanations. We wanted to see whether a classifier could pick up these differences.

We used 11,640 responses from six families: GPT, Claude, Gemini/PaLM, Llama-2, Mistral and Qwen. The main question is to predict the family from the response. We also tried giving the classifier the prompt, moving it to a different type of task, and removing parts of the text that might carry the signal.

Our starting guess: a model may be easier to recognize by its usual wording than by the way it develops an entire answer. That is a fairly simple guess, but it gives us something to test. A CNN looks for short patterns, whereas an LSTM reads the input in order and makes its decision from a final state. If the CNN does better, we can then ask whether other experiments point in the same direction.

What happened: the CNN reached 52.0% accuracy and the LSTM reached 31.0%, averaged over three seeds. Logistic regression on tokens and token pairs reached 52.5%. Random guessing among six families gives

$$P(\text{correct}) = \frac{1}{6} \approx 16.7\%.$$

So there is something to recognize, although we are still wrong on many responses. The interesting part is what that something might be.

This report follows the experiments in that order. We use the course ideas of supervised classification, CNNs, recurrent networks, cross-entropy training, and clustering, but treat the results as an exploration of this particular dataset.

2. Building the dataset

Source: we used the public *arena-human-preference-55k* dataset. An Arena record contains one prompt and two model responses, together with their model names and a preference vote. We only need the prompt, responses and names; the vote does not enter the classifier. The dataset card is listed in the references.

Keeping the first turn: we take the first prompt and first response from each conversation. We remove prompts shorter than five characters, responses shorter than twenty characters, and duplicate prompt–response pairs. We group versions by their model names. GPT-3.5 and GPT-4 go into GPT, Gemini and PaLM into the Google family, CodeLlama into Llama-2, and Mixtral into Mistral. We leave out families without enough data and ambiguous community fine-tunes.

Balancing tasks: a model might otherwise be recognized just because it has more coding examples. We assign a task from the prompt using keyword rules, then keep the same counts for each family:

$$340 \text{ coding} + 190 \text{ math} + 110 \text{ writing} + 1300 \text{ QA} = 1940.$$

Multiplying by six gives 11,640 responses. These task labels are rough. For example, "how many" can occur in a general question without making it a mathematical problem. We check this later.

Splitting by prompt: a deterministic hash assigns a prompt to training, validation or test with nominal proportions 70/15/15. All retained responses with the same prompt go into the same split. This avoids training on one answer to a prompt and testing on another answer to that exact prompt. The proportions are approximate because prompt groups have different sizes.

Table 1: Response counts after splitting by prompt.

Family	Train	Validation	Test	Total
GPT	1355	308	277	1940
Claude	1352	294	294	1940
Gemini/PaLM	1378	280	282	1940
Llama-2	1326	313	301	1940
Mistral	1353	286	301	1940
Qwen	1325	315	300	1940
Total	8089	1796	1755	11640

The final dataset contains 27 model versions. Appendix A lists the versions and their counts. The median response has 1,042 characters, and the median prompt has 84. The included dataset contains actual answers, including answers to the same prompt from different families.

3. Models and training

(a) Representing the text

We lower-case the text and split it into words and punctuation. A line break becomes a separate `<nl>` token. We keep punctuation and Markdown markers because removing them at the start would throw away a possible clue before testing whether it matters.

The neural vocabulary is built from training text only. It considers the 30,000 most frequent tokens, requires at least two occurrences, and adds special tokens. Each input token gets a learned vector in $\mathbb{R}^{128}$. The response-only networks read at most 200 tokens. About 44.0% of the responses fit within this limit; for the rest, the networks see only the beginning.

(b) Two simple baselines

Before comparing the networks, we use Bernoulli Naive Bayes and logistic regression. Their features indicate whether a token or consecutive token pair occurs in the text. The vocabulary contains at most 100,000 features, each observed in at least two training documents.

Naive Bayes estimates feature probabilities separately for each family, with a smoothing value selected on validation data. Logistic regression learns one weight for each feature and family. These baselines use the full text, so their comparison with a 200-token network is useful but does not isolate architecture alone.

(c) CNN

A phrase such as "here is" can appear at different positions. Convolution gives us a way to look for a short pattern across the response and keep its strongest match.

Architecture: embedding vectors → three one-dimensional convolutions with window sizes 3, 4 and 5, each with 100 filters → ReLU → max-pooling over positions → a 300-dimensional vector → a linear layer with six outputs. Padding windows are masked before pooling.

(d) LSTM

The LSTM reads the input from left to right. We use one layer with 128 hidden units, take the hidden state at the last real token, and pass it to a linear layer with six outputs. This gives the classifier a chance to use sequential information, but also makes it compress the prefix into one vector.

We should not expect this small LSTM to settle the question of whether whole-answer organization matters. Both networks see only a prefix, and the LSTM may simply be harder to train with our settings.

(e) Training and evaluation

For logits $z_1, \ldots, z_6$ and true label y, the training loss for one response is

$$\ell(z, y) = -\log \frac{e^{z_y}}{\sum_{j=1}^{6} e^{z_j}}.$$

We use Adam with learning rate 0.001, batch size 64, and five epochs. After each epoch we measure validation accuracy and save the best state. We report its test accuracy, together with macro F1 and per-family results. The response-only comparison uses seeds 0, 1 and 2; the other neural experiments use seed 0.

4. RQ1: Can we recognize the family?

Table 2: Response-only test results. Neural accuracy is mean ± sample standard deviation in percentage points over seeds 0, 1 and 2.

Classifier	Accuracy	Macro F1
Naive Bayes	39.7%	0.367
Logistic regression	52.5%	0.522
CNN	52.0% ± 1.7	0.517
LSTM	31.0% ± 1.2	0.289

Yes, to some extent. All four classifiers beat 16.7%. The CNN is much better than our LSTM, but the simpler logistic regression is about as accurate as the CNN. Naive Bayes sits between them. This already makes short lexical clues worth investigating: a classifier that checks token and token-pair presence can do quite well without representing the full sequence.

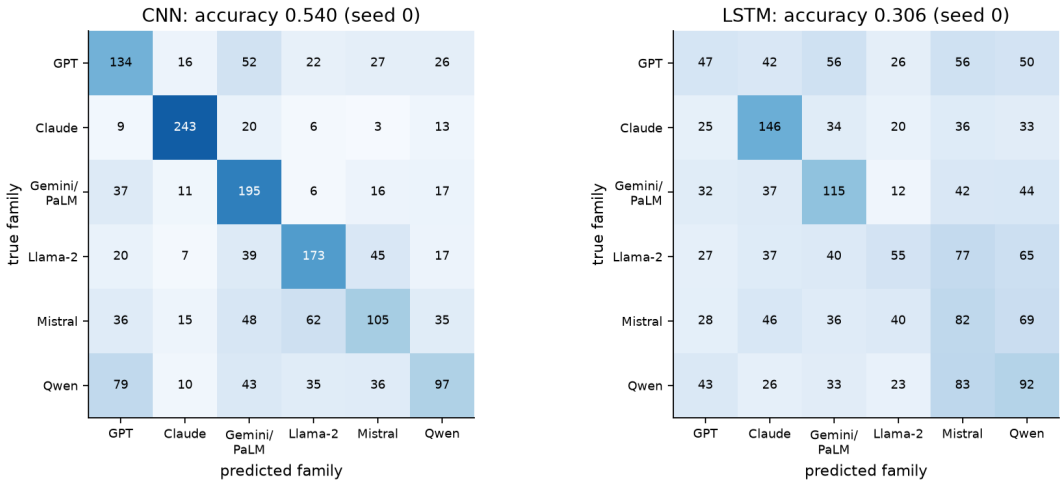


Figure 1: Confusion matrices for seed 0. Rows are true families and columns are predictions.

Some families are easier: Claude has the highest CNN F1, about 0.81; Mistral has the lowest, about 0.34. In the seed-0 CNN, the most common two-way confusion is Llama-2 versus Mistral, with 107 responses in total. So "recognizing the family" is not equally easy for every family.

Table 3: F1 by family. Neural values are averaged over three seeds.

Family	NB	LR	CNN	LSTM
GPT	0.254	0.375	0.419	0.205
Claude	0.611	0.856	0.809	0.485
Gemini/PaLM	0.393	0.638	0.565	0.307
Llama-2	0.421	0.527	0.555	0.290
Mistral	0.244	0.362	0.339	0.200
Qwen	0.281	0.373	0.413	0.244

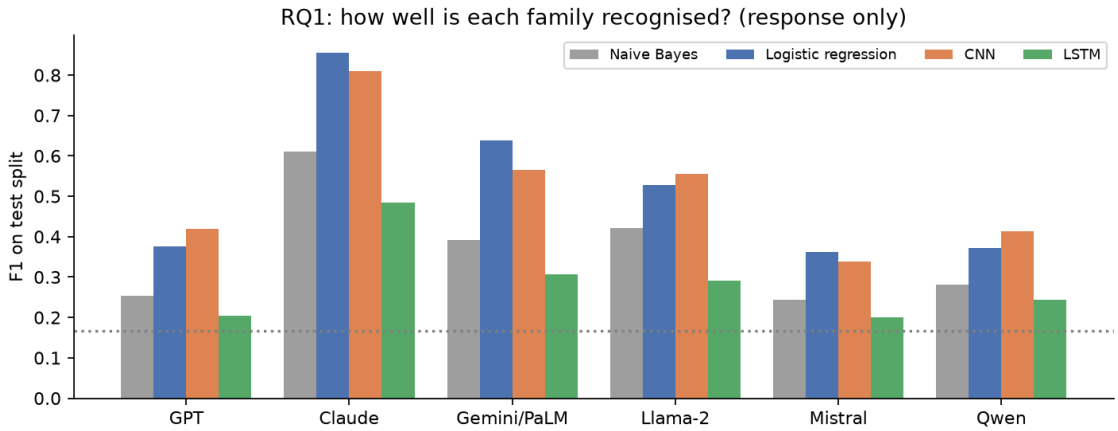


Figure 2: F1 scores for each family. CNN and LSTM values average three seeds.

Why did the LSTM do worse? One possible reason is that useful openings are easy for max-pooling to retain but have to survive many steps before the LSTM's final state. Another is simply that the LSTM needs more training or a different setup. At epoch five, the seed-0 CNN's training accuracy is 94.8%, while the LSTM's is 42.1%. The LSTM is not even fitting the training set nearly as well, so we cannot attribute the whole gap to the kind of information it reads.

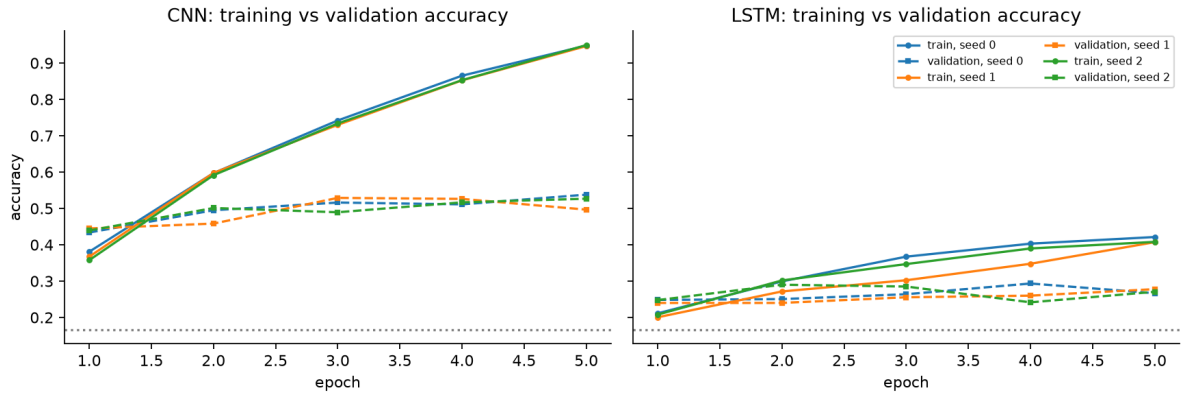


Figure 3: Training and validation accuracy for the three seeds. Each reported test result uses the epoch selected by validation accuracy.

The gap between training and validation also reminds us that learning to recognize training responses is easier than recognizing new ones. More epochs alone may not solve the problem.

5. RQ2: Does the prompt help?

Table 4: Input comparison; neural results use seed 0.

Classifier	Prompt	Response	Both
NB	17.0%	39.7%	38.2%
LR	16.8%	52.5%	52.1%
CNN	18.7%	54.0%	50.5%
LSTM	17.0%	30.6%	31.4%

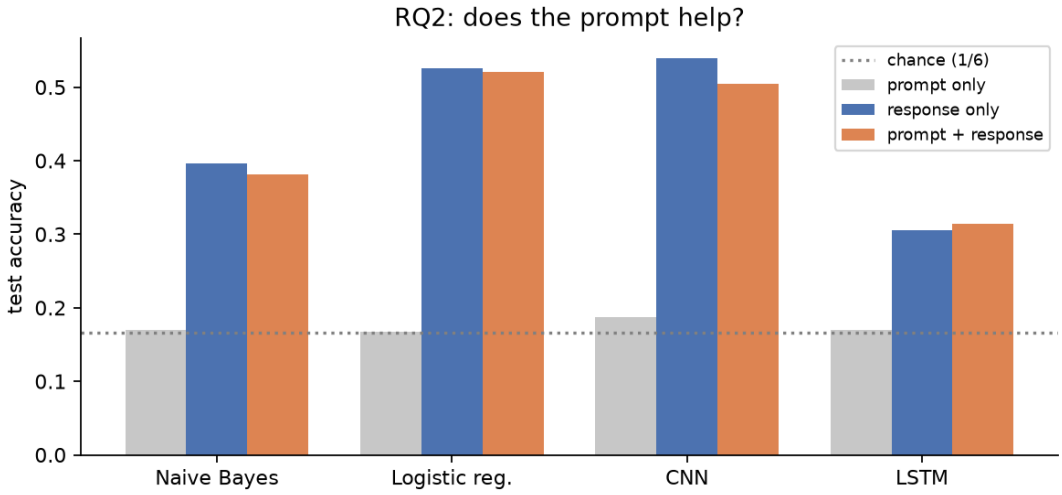


Figure 4: Prompt-only, response-only and combined inputs. The horizontal line is uniform random guessing.

Prompt alone: the results are close to chance. The largest is the CNN's 18.7%, compared with 16.7% for uniform guessing. The prompt therefore gives little useful family information in this experiment. Arena is helpful here because the same prompt can receive answers from different models. Still, this result does not prove that prompt and family are perfectly independent after filtering and sampling.

Prompt plus response: the CNN drops from 54.0% to 50.5%, while the LSTM changes from 30.6% to 31.4%. There is no clear benefit across models. These are seed-0 comparisons; the CNN's three-seed average in RQ1 is 52.0%.

There is also a simple catch. The combined neural input has the same 200-token budget, and the prompt can take up to 128 tokens plus a separator. A long prompt leaves only 71 response tokens. Adding context also removes part of the answer. A fairer follow-up would give the response its own fixed budget and add the prompt separately. For now, the conclusion is only that our current way of adding the prompt did not consistently help.

6. RQ3: Does recognition carry over to another task?

If the classifier is recognizing a recurring way of writing, it might still work on a task type it has not been trained on. If it mainly recognizes how a model writes code, moving it to writing or math could make it fail.

We train on selected task types, choose the epoch using validation examples from those types, and evaluate on an unseen task. "Source" below means held-out examples of the training task types, not the training examples themselves. The all-task reference uses the same target test rows.

Table 5: CNN transfer results (seed 0). Source is held-out accuracy on training task types; All tasks is the reference model on the same target test rows.

Train tasks	Target	n	Source	Target	All tasks
math	code	291	37.2%	26.8%	54.3%
QA	code	291	51.6%	43.6%	54.3%
write	code	291	48.0%	30.2%	54.3%
code	math	183	38.8%	31.7%	47.0%
code + math + write	QA	1183	44.6%	41.8%	54.7%
code + math + QA	write	98	51.3%	52.0%	57.1%
code	write	98	38.8%	32.7%	57.1%
math	write	98	37.2%	28.6%	57.1%

Table 6: LSTM transfer results (seed 0). Source is held-out accuracy on training task types; All tasks is the reference model on the same target test rows.

Train tasks	Target	n	Source	Target	All tasks
math	code	291	18.0%	18.9%	27.5%
QA	code	291	30.0%	22.7%	27.5%
write	code	291	23.5%	11.7%	27.5%
code	math	183	21.0%	20.8%	25.1%
code + math + write	QA	1183	25.9%	26.0%	32.5%
code + math + QA	write	98	31.6%	29.6%	26.5%
code	write	98	21.0%	23.5%	26.5%
math	write	98	18.0%	19.4%	26.5%

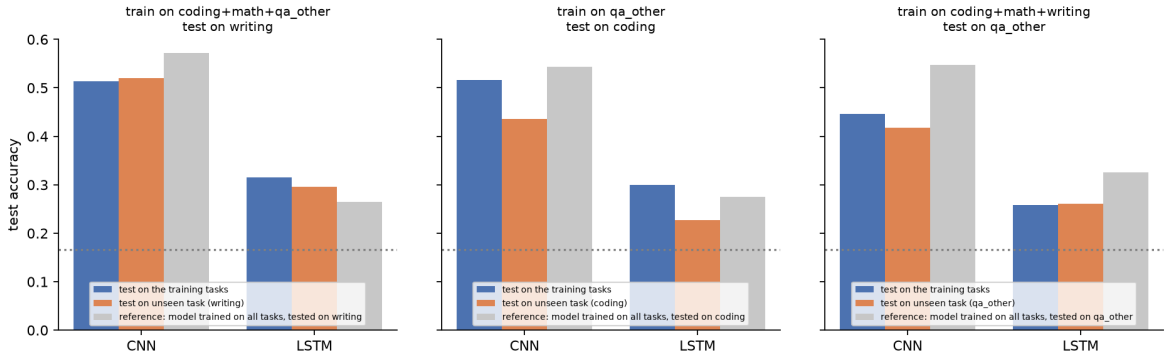


Figure 5: Three transfer settings, comparing source tasks, the unseen target task, and an all-task reference.

The CNN remains above chance in every setting, but the size of the drop varies. QA to coding reaches 43.6%, whereas math to coding reaches 26.8%. Training on coding, math and QA and testing on writing gives 52.0%, close to its source-task score of 51.3%. That looks promising, but the writing test set has only 98 responses.

Our guess: some clues travel across tasks. Openings, lists and common turns of phrase need not belong to only one task. But a task change also changes the amount of training data. A network trained only on math or writing gets far fewer examples than one trained on QA, so these results mix transfer difficulty with sample size.

Table 7: CNN recall averaged equally across the eight transfer settings.

Family	**Source recall**	**Target recall**
GPT	0.227	0.140
Claude	0.676	0.552
Gemini/PaLM	0.477	0.399
Llama-2	0.450	0.400
Mistral	0.378	0.275
Qwen	0.428	0.351

Claude retains the highest average target recall, about 0.55. GPT is much lower, about 0.14. This makes us wonder whether an especially recognizable old Claude style is carrying much of its transfer performance.

Checking the task labels: we reviewed 50 training prompts sampled by rule label: 13 coding, 13 math, 12 writing and 12 QA. Initial judgments were discussed with an AI assistant and then checked row by row by a team member. This was an assisted review, not a blind independent annotation.

Table 8: Task-label check: reviewed labels in rows, rule labels in columns (50 training prompts).

Reviewed / rule	**code**	**math**	**QA**	**write**
code	11	0	0	0
math	0	5	1	0
QA	2	7	11	1
write	0	1	0	11

There are 38 agreements out of 50, or 76.0%. Only 5 of the 13 prompts labelled math by the rules were confirmed as math. The other rule-label agreement rates are 84.6% for coding and 91.7% for both writing and QA. These numbers describe this stratified review sample; they are not an estimate of overall test-label accuracy. In particular, we should be cautious about what the math-transfer results mean.

6.1 Trying newer models

Hongchang's newer-model check gives us a useful complication. The classifiers were trained on older Arena responses. What happens when a newer version of the same family answers?

The saved collection contains 19 responses to five shared coding prompts from the test split, generated on 11 October 2026: five each from GPT-4.1 mini, Claude Haiku 4.5 and Gemini 2.5 Flash, and four from Qwen3-235B. The generation temperature was 0.7 with a 1,024-output-token limit. We apply the seed-0 CNN and LSTM without retraining.

Table 9: Correct-family rates on 19 responses to five coding prompts; no retraining.

Newer model	n	CNN	LSTM
Claude Haiku 4.5	5	0.0%	0.0%
Gemini 2.5 Flash	5	80.0%	60.0%
GPT-4.1 mini	5	80.0%	0.0%
Qwen3-235B	4	0.0%	0.0%

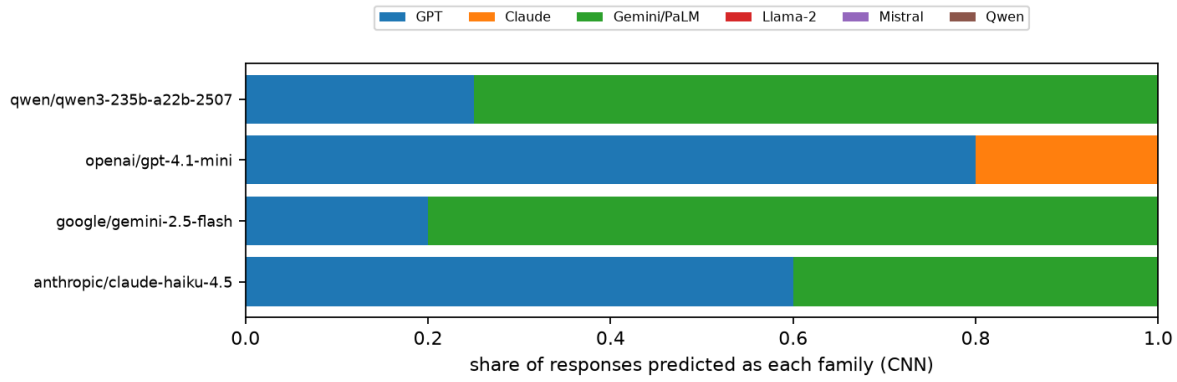


Figure 6: The families assigned by the CNN to the newer responses.

The CNN gets 8 of 19 right, or 42.1%; the LSTM gets 3 of 19 right, or 15.8%. Neither recognizes any of the five Haiku 4.5 responses or the four Qwen3 responses. That is striking next to old Claude's strong result in RQ1.

Does this weaken our short-phrase guess? It makes us less confident that we found a stable fingerprint of a *family*. It does not necessarily contradict the guess about wording and formatting. If those habits change between versions, a classifier relying on them could become confused. In that sense, the failure fits the guess, but it is not a separate proof of it.

For example, the saved whole-response measurements give old Claude about 0.02 bold spans per answer and Haiku 4.5 about 6.2; old Qwen has about 0.39 and Qwen3 about 20.5. This suggests a possible change in presentation. However, these averages compare

different task and length distributions, and the classifiers only read the first 200 tokens. We cannot say that bold text caused the wrong predictions.

Eight responses reached the generation limit, including all five Gemini responses. Using only a prefix does not make that limitation disappear, especially when we inspect whole-response formatting. More basically, there are only five distinct prompts here. We would need more prompts and a controlled change to the formatting to tell apart version changes, task effects and surface style.

7. RQ4: What might the classifiers be recognizing?

(a) A few things we can measure

We measure 14 features: word and sentence counts, words per sentence, type–token ratio, average word length, line breaks, bullets, numbered lines, headings, bold spans, code blocks, common affirmative openings, refusal phrases, and first-person words per thousand words.

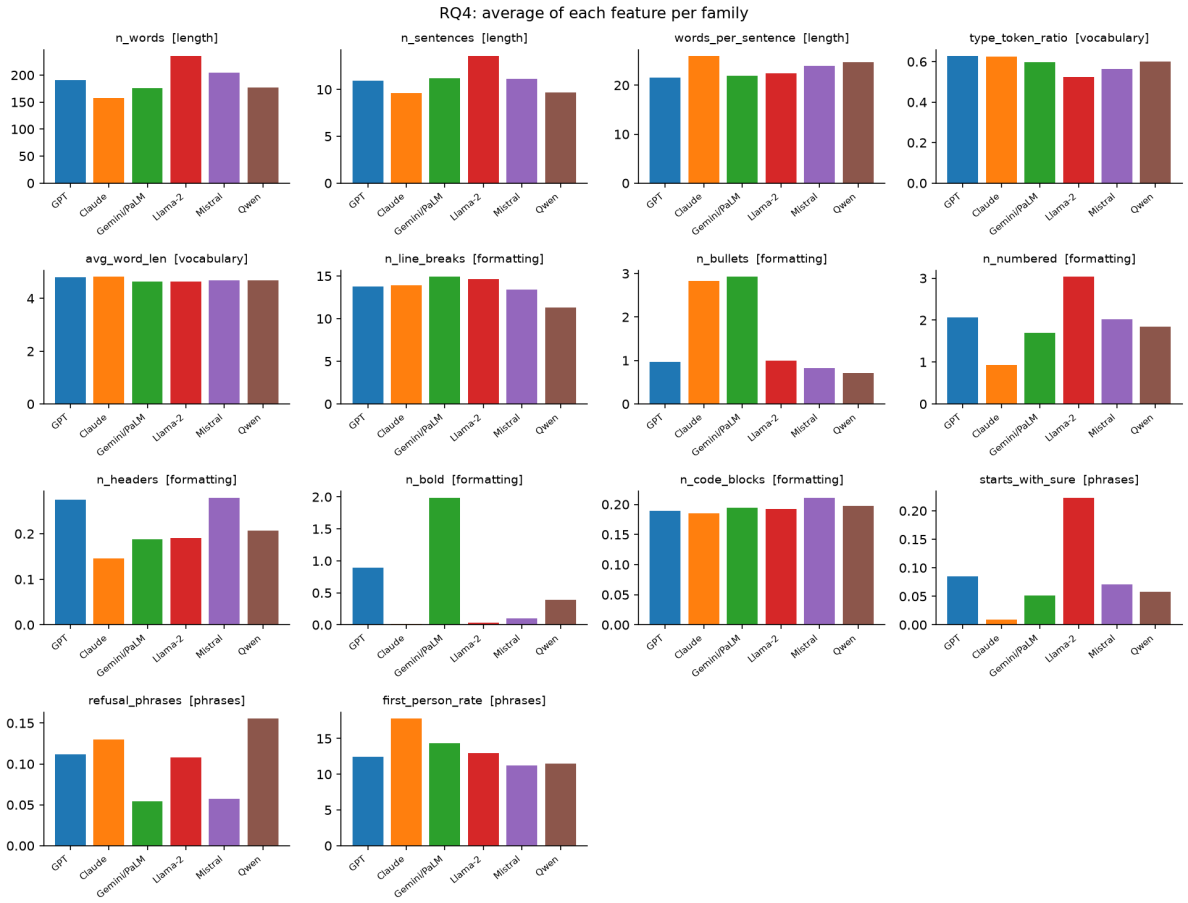


Figure 7: Average measured features by family. These summarize full responses.

Some differences are visible. Old Gemini/PaLM uses more bold text. Old Claude uses bullets but very little bold text. Llama-2 has longer responses and begins with an affirmative expression such as “Sure” relatively often. These are ordinary writing habits, not a measurement of reasoning quality.

Table 10: Logistic regression on measured features. These exploratory runs used pooled scaling; see the limitation in the text.

Features	**Count**	**Accuracy**
all 14 features	14	37.4%
length only	3	24.4%
vocabulary only	2	21.9%
formatting only	6	30.8%
phrases only	3	24.5%

Formatting features alone reach 30.8%, compared with 24.4% for length, 21.9% for vocabulary measures, and 24.5% for the phrase group. All 14 together reach 37.4%. Among these small feature groups, formatting seems worth paying attention to.

These feature experiments have a limitation: the script computes its scaling mean and standard deviation over the full dataset before fitting on training plus validation rows. That lets the scaling see test-distribution information. We keep these saved numbers as exploratory results, rather than treating them as a clean performance comparison with RQ1. A repeat should fit the scaling on training data only.

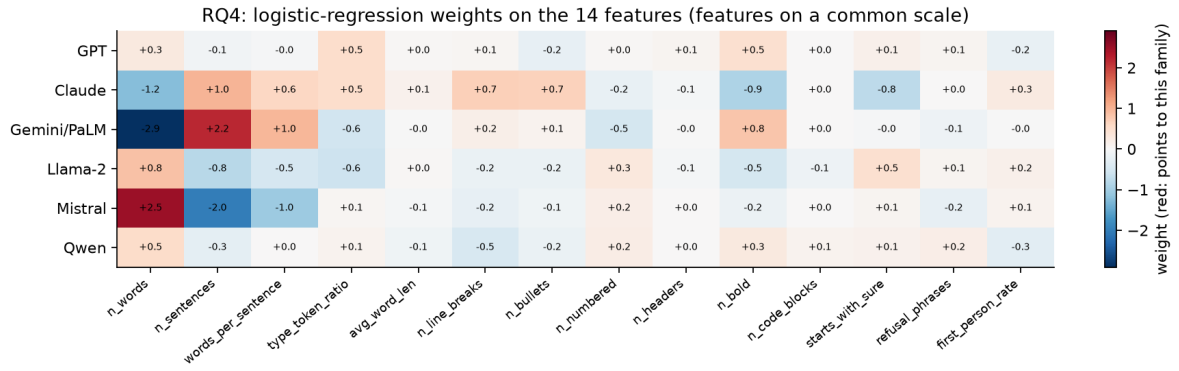


Figure 8: Weights of logistic regression on the scaled features. Signs indicate an association in this fitted model, not a causal effect.

(b) Looking at actual tokens

Table 11: Response-only logistic regression. <nl> denotes a line break; punctuation is tokenized separately.

Family	**Ten highest-weight tokens or token pairs**
GPT	assist; equals; remember; ai ,; request .; with that; have three; these; >; generous

Family	Ten highest-weight tokens or token pairs
Claude	: <nl>; here is; <nl> -; i do; apologize; apologize ,; i apologize; so; * so; actually
Gemini/PaLM	* *; : *; *; <nl> -; * <nl>; sure ,; <nl> here; <nl> *; ‘; ‘ ‘
Llama-2	sure !; ! here; <nl> <nl>; 52; !; sure; so ,; , here; just an; <nl> 1
Mistral	brand; thought; note; antiseptic; lonely; , thought; note :; , so; joke for; thought ,
Qwen	sorry ,; sorry; по; language model; m sorry; be february; true ,; model ,; , ; ai language

The high-weight features include line breaks, punctuation pairs and familiar expressions. Some are lexical or language clues too. A large coefficient is an association in this fitted classifier; it does not mean the phrase uniquely belongs to that family. Still, the list is a more concrete clue than just saying that one model has a different “style.”

(c) Removing possible clues

Table 12: Retrained models under each response transformation (seed 0).

Response	CNN	LSTM
full text	54.0%	30.6%
formatting removed	43.4%	27.7%
first 64 tokens only	47.0%	31.6%
word order shuffled	35.8%	26.2%

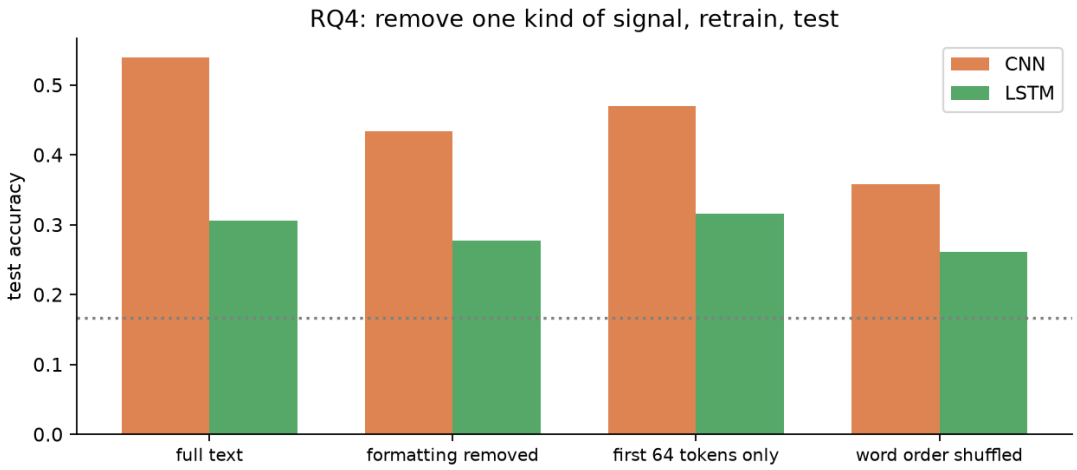


Figure 9: Response transformations applied before retraining and evaluation. All runs use seed 0.

Removing formatting lowers the CNN from 54.0% to 43.4%. Shuffling lowers it to 35.8%, while keeping only the first 64 tokens still gives 47.0%. The opening part of an answer seems to contain a fair amount of useful information.

There is a catch in the shuffling experiment too: the script shuffles the whole response *before* taking the first 200 tokens. It changes both order and which tokens the network gets to see. Removing formatting can also make more words fit inside 200 tokens. So these are useful perturbations, but not perfectly isolated tests of word order or format.

(d) Our working guess

The CNN–LSTM gap first suggested the idea, but we would not rely on that gap alone. Logistic regression on tokens and token pairs also works well, and changing the order or formatting hurts the CNN. Together, these results suggest that **our classifiers recognize repeated wording and presentation habits more easily than they recognize how an entire answer is organized**.

That is the conclusion we would like to keep, with its scope made clear. We have not measured the model's internal reasoning or "behavioral logic." We have measured patterns in its text. The weak LSTM, different input lengths and imperfect ablations leave room for a stronger sequence model to find something we missed.

The newer-model check adds another part to the story. Perhaps our classifier has learned "how these versions tend to sound" rather than "what makes this family itself." Haiku 4.5 being missed after old Claude was easy to identify makes that a reasonable next question. It weakens the idea of a permanent family signature, while leaving the local-habit explanation plausible.

(e) Three simple follow-up guesses

Maybe the opening gives the model away. We could compare the first, middle and last 64 tokens, keeping their lengths equal. If the first part wins, we could remove greetings and standard opening phrases and try again.

Maybe changing the layout changes the prediction. We could keep the words fixed and rewrite only the line breaks, bullets and bold markers. If an old Claude answer starts being labelled Google after that, we would have a much cleaner example of a formatting shortcut.

Maybe a newer version sounds like a different old family. We could give old and new versions the same larger set of prompts, normalize their formatting, and compare the errors. If Haiku is still assigned to GPT after normalization, then formatting alone is not a good explanation.

These are guesses that came from the results, not experiments we have already completed. They would help turn the current explanation into something more directly testable.

8. What happens without labels, or with less data?

(a) K-means

We also run K-means with $k = 6$ on the 14 measured features. If family identity were the main source of variation in those features, perhaps the clusters would roughly line up with the families.

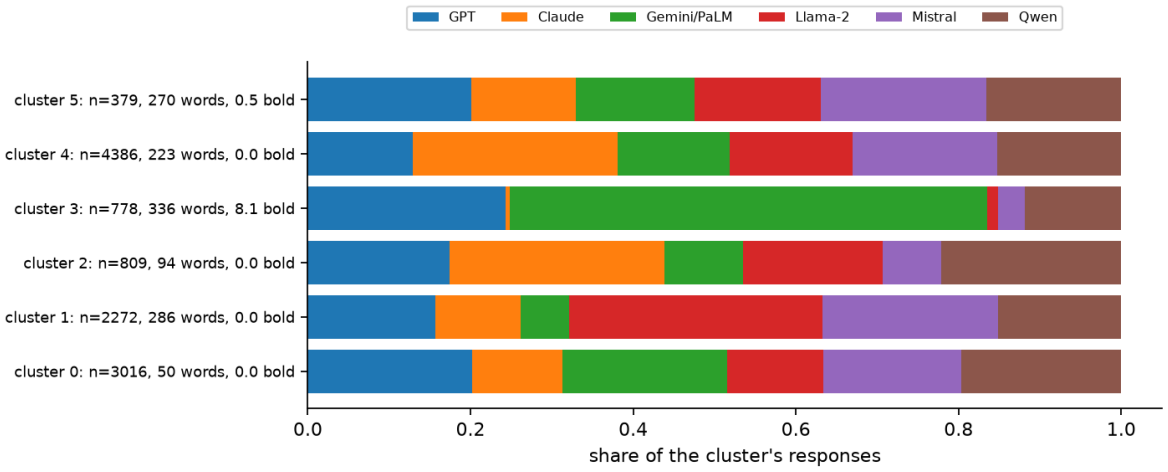


Figure 10: Family composition of the clusters.

Cluster purity is 0.27, compared with about 0.18 after randomly permuting cluster assignments while preserving their sizes. The clusters tend to separate kinds of responses: short answers, code, long text, or heavy formatting. One bold-heavy cluster has 58.7% Gemini/PaLM responses, but most clusters mix families.

This does not show that unsupervised learning cannot recover the families. It only shows that these 14 features and this clustering setup do not separate them very well. A simple guess is that task and response form are larger sources of variation than family identity in this feature space.

(b) Less training data

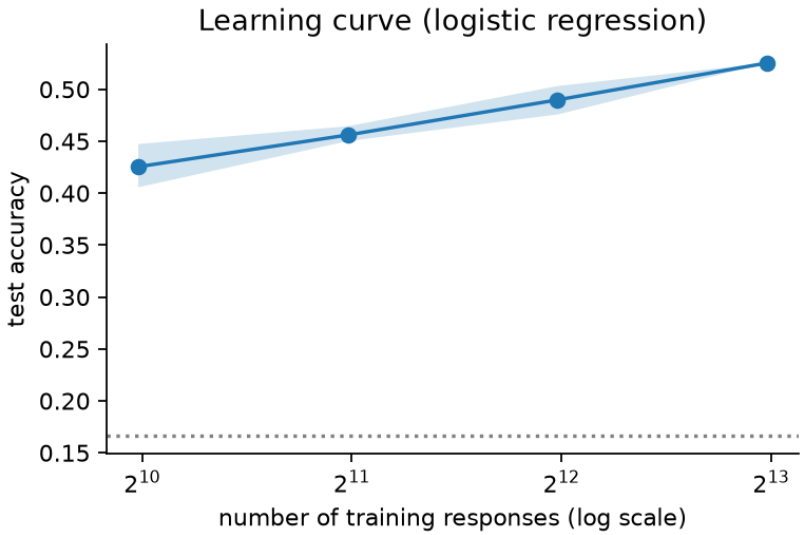


Figure 11: Logistic-regression accuracy as the training sample grows.

Table 13: Logistic-regression learning curve; subset results average three seeds.

Training responses	**Accuracy**
1,011	42.5%
2,022	45.6%
4,044	49.0%
8,089	52.5%

Accuracy keeps increasing over the tested sizes. More examples might therefore help, although the curve does not tell us how far that improvement would continue. It would also be useful to match the CNN and LSTM on several training budgets before deciding that one architecture is inherently better for this task.

9. What we take from this

The first result is straightforward: these responses contain enough information to identify the family above chance. The more interesting result is how cheaply some of that information can be used. Logistic regression is about as accurate as the CNN, and our LSTM is substantially weaker. This makes short lexical and formatting habits a sensible starting explanation.

We initially expected that recognizing a family might mean recognizing something fairly stable about it. The newer-model check makes that expectation less comfortable. Old Claude is easy to recognize; Haiku 4.5 is missed in all five cases. The sample is small, but the contrast is enough to make us ask whether we are recognizing a particular generation of models instead of a family across time.

The project also changed how we read an accuracy number. The task labels are noisy, the writing test set is small, the networks only see prefixes, and several comparisons change more than one thing at once. Those details do not erase the findings. They tell us which next experiment would be worth doing.

Our current view is that repeated phrases and presentation habits account for a useful part of recognition in this setup. Whether whole-answer organization adds much more, and whether any of the signal survives a substantial version change, remain open questions. Those would be more interesting to investigate next than simply making the current accuracy number a little higher.

AI assistance. We carried out the project and checked the results. We used AI as a tutor to discuss the methods, work through questions about code and interpretation, and improve the presentation of the report. The task-label review described in Section 6 was checked by a team member after the assisted discussion.

References

1. CMPSC 448 lecture notes: supervised and unsupervised learning, deep learning architectures, language-model pretraining, instruction tuning, preference learning, and optimization.

2. LMArena. *Arena Human Preference 55K*, dataset and dataset card. `https://huggingface.co/datasets/lmarena-ai/arena-human-preference-55k`.

3. Zheng, L. et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS. `https://arxiv.org/abs/2306.05685`.

4. Kim, Y. (2014). *Convolutional Neural Networks for Sentence Classification*. EMNLP, 1746–1751. `https://aclanthology.org/D14-1181/`.

5. Hochreiter, S. and Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780.

Appendix A: Model versions

Table 14: The 27 model versions in the balanced dataset.

Family	**Model versions and response counts**
GPT	gpt-4-1106-preview (463); gpt-3.5-turbo-0613 (432); gpt-4-0613 (401); gpt-4-0314 (245); gpt-3.5-turbo-1106 (218); gpt-3.5-turbo-0314 (69); gpt-4-0125-preview (68); gpt-3.5-turbo-0125 (44)
Claude	claude-2.1 (690); claude-instant-1 (489); claude-1 (442); claude-2.0 (319)
Gemini/PaLM	palm-2 (824); gemini-pro-dev-api (589); gemini-pro (527)
Llama-2	llama-2-70b-chat (697); llama-2-13b-chat (538); llama-2-7b-chat (365); codellama-34b-instruct (340)
Mistral	mistral-medium (783); mixtral-8x7b-instruct-v0.1 (779); mistral-7b-instruct (355); mistral-7b-instruct-v0.2 (23)
Qwen	qwen-14b-chat (1023); qwen1.5-72b-chat (524); qwen1.5-7b-chat (204); qwen1.5-4b-chat (189)

Appendix B: Feature and cluster details

Table 15: Mean feature values. Google denotes Gemini/PaLM.

Feature	GPT	Claude	Google	Llama-2	Mistral	Qwen
n words	191.09	158.14	175.66	235.63	204.94	177.52
n sentences	10.91	9.62	11.18	13.59	11.14	9.66
words per sentence	21.61	26.00	21.99	22.45	24.00	24.70
type token ratio	0.63	0.63	0.60	0.52	0.57	0.60
avg word len	4.80	4.82	4.64	4.63	4.69	4.68
n line breaks	13.75	13.91	14.92	14.61	13.37	11.29
n bullets	0.96	2.83	2.93	1.00	0.82	0.71
n numbered	2.07	0.94	1.70	3.03	2.01	1.85
n headers	0.27	0.15	0.19	0.19	0.28	0.21
n bold	0.89	0.02	1.99	0.04	0.10	0.39
n code blocks	0.19	0.19	0.19	0.19	0.21	0.20
starts with sure	0.09	0.01	0.05	0.22	0.07	0.06
refusal phrases	0.11	0.13	0.05	0.11	0.06	0.16
first person rate	12.43	17.82	14.33	12.97	11.26	11.49

Table 16: Family counts in the six K-means clusters.

Cluster	GPT	Claude	Google	Llama-2	Mistral	Qwen
0	610	333	612	357	511	593
1	356	238	135	709	489	345
2	141	214	78	139	58	179
3	189	4	457	10	26	92
4	568	1102	603	666	779	668
5	76	49	55	59	77	63

Table 17: Mean measured features in each cluster.

Cluster	Words	Line breaks	Bullets	Bold	Code	Refusals
0	49.94	1.14	0.06	0.01	0.02	0.00
1	285.52	18.37	1.16	0.01	0.17	0.02
2	93.89	3.24	0.17	0.02	0.00	1.39
3	335.94	25.65	6.68	8.08	0.21	0.02
4	223.05	16.84	2.13	0.02	0.24	0.00
5	270.19	45.34	1.09	0.51	1.65	0.01

Appendix C: All network runs

The tables below retain the saved results from all 32 runs. In task names, code means coding, write means writing, and QA means the general-question group. Minutes are the recorded elapsed training times, which are affected by concurrent jobs.

Table 18: All 16 CNN runs. Input means prompt-only; both concatenates prompt and response.

Input	Train	Test	Ablation	Seed	Acc.	Ep.	Min.
both	all	all	none	0	50.5%	3	32.7
input	all	all	none	0	18.7%	3	24.4
output	all	all	none	0	54.0%	5	32.7
output	all	all	none	1	50.9%	3	32.7
output	all	all	none	2	51.1%	5	33.3
output	all	all	shuffle	0	35.8%	5	33.4
output	all	all	strip format	0	43.4%	5	33.5
output	all	all	trunc64	0	47.0%	4	14.7
output	code + math + QA	write	none	0	52.0%	5	31.4
output	code + math + write	QA	none	0	41.8%	3	16.4
output	code	math	none	0	31.7%	4	4.4
output	code	write	none	0	32.7%	4	4.6
output	math	code	none	0	26.8%	5	2.7
output	math	write	none	0	28.6%	5	1.2
output	QA	code	none	0	43.6%	5	24.8
output	write	code	none	0	30.2%	5	0.8

LSTM runs

Table 19: All 16 LSTM runs. Input means prompt-only; both concatenates prompt and response.

Input	Train	Test	Ablation	Seed	Acc.	Ep.	Min.
both	all	all	none	0	31.4%	5	5.0
input	all	all	none	0	17.0%	2	9.8
output	all	all	none	0	30.6%	4	5.1
output	all	all	none	1	32.4%	5	5.0
output	all	all	none	2	30.0%	2	5.0
output	all	all	shuffle	0	26.2%	2	5.3
output	all	all	strip format	0	27.7%	4	5.2
output	all	all	trunc64	0	31.6%	4	3.4
output	code + math + QA	write	none	0	29.6%	5	5.0
output	code + math + write	QA	none	0	26.0%	5	3.4
output	code	math	none	0	20.8%	5	1.0
output	code	write	none	0	23.5%	5	1.1
output	math	code	none	0	18.9%	5	0.7
output	math	write	none	0	19.4%	5	0.2
output	QA	code	none	0	22.7%	2	3.8
output	write	code	none	0	11.7%	3	0.1